SOFTWARE ENGINEERING LAB

Paper Code: CIC-357

PRACTICAL FILE

Project Title

CREDIT CARD FRAUD DETECTION SYSTEM

Submitted By	
Enrollment No.	
Branch / Semester	B.Tech, Semester 5
Group Members	
Submitted To	
Institute	

INDEX

Exp. No.	Title of Experiment	Date	Sign
1	Problem statement for the Credit Card Fraud Detection System		
2	Requirement analysis and Software Requirement Specification (SRS)		
3	Function-oriented diagrams: Data Flow Diagram (Levels 0, 1, 2) and Structure Chart		
4	Entity Relationship (ER) diagram		
5	User's view analysis: Use Case diagram		
6	Structural view: Class diagram and Object diagram		
7	Behavioral view: State-chart diagram and Activity diagram		
8	Behavioral view: Sequence diagram and Collaboration diagram		
9 (CBS 1)	Implementation view: Component diagram		
10 (CBS 2)	Environmental view: Deployment diagram		

Experiment 1: Problem Statement

Aim:- To write down the problem statement for a suggested system of relevance: the Credit Card Fraud Detection System.

Description

Credit card fraud is the unauthorised use of a payment card, or of its details, to obtain goods, services or cash. As card-not-present (online) payments, digital wallets and contactless payments have grown, fraudsters have more ways to steal card data through phishing, skimming, data breaches and account takeover, and to use it before the real cardholder notices. Banks and payment processors therefore need an automated way of deciding, within a fraction of a second, whether a card transaction is genuine or suspicious.

A Credit Card Fraud Detection System is a software platform that receives every card transaction as it happens, studies it against the cardholder's normal behaviour and known fraud patterns, and assigns it a fraud risk score. Depending on that score the transaction is approved, held for verification or declined, and suspicious cases are sent to a human fraud analyst for review. The confirmed outcomes are then fed back so that the detection model keeps learning as fraud tactics change.

Software Description

The proposed system is a web-based, service-oriented application. Its detection core is a supervised machine-learning classifier (for example Random Forest or XGBoost) trained on labelled historical transactions and combined with a configurable rule engine. It is exposed to payment gateways through a REST API and to fraud analysts and administrators through a web dashboard. A good fraud detection system should provide the following features:

- **Real-time transaction scoring:** every incoming transaction is scored within a few hundred milliseconds so that the payment flow is not delayed.
- **Behavioural feature engineering:** transaction velocity (number of transactions in the last minutes/hours), unusual amounts, new merchant or country, and distance or time between consecutive transactions.
- **Hybrid detection:** a machine-learning model to catch unseen patterns plus rules (for example, blocked countries or amount limits) for known fraud scenarios.
- **Alert and case management:** automatic alert creation, analyst assignment, review workflow and final verdict (fraud / false positive).
- **Cardholder notification and step-up verification:** SMS or e-mail alerts and confirmation of suspicious transactions by the cardholder.
- **Continuous learning:** confirmed fraud and false-positive labels are stored and used for scheduled model retraining and evaluation.
- **Reports and dashboards:** fraud rate, detection rate, false-positive rate, alert backlog and analyst productivity.

Problem in Existing System

- **Class imbalance and hidden fraud:** genuine transactions vastly outnumber fraudulent ones (in the widely used public ULB/Kaggle card dataset only 492 of 284,807

transactions, about 0.17%, are fraud). Simple approaches therefore miss most fraud or generate too many false alarms.

- **Rigid rule-based checks:** static rules such as fixed amount limits are easily learned and bypassed by fraudsters, and they must be updated manually every time a new pattern appears.
- **False positives:** legitimate purchases that are wrongly declined annoy customers and cost the bank revenue and trust.
- **Delayed detection:** batch or manual review often finds fraud only after the money has already moved, when it can only be recovered through chargebacks.
- **Concept drift:** fraud tactics change quickly, so a model that is never retrained becomes less accurate over time.
- **Data privacy constraints:** real card data is sensitive and regulated, which makes it difficult to share data for building and testing detection systems.

Requirements

- **Security and privacy:** card numbers must be tokenised or masked, all traffic encrypted, and access limited by role in line with PCI DSS guidance.
- **Low latency and high throughput:** scoring must complete fast enough to sit inside the payment authorisation flow and must scale during peak shopping periods.
- **Accuracy:** high recall (fraud caught) while keeping precision reasonable so genuine customers are not blocked; evaluated with precision, recall, F1-score and PR-AUC instead of plain accuracy.
- **Integration capability:** standard REST/JSON interfaces so that the system can be plugged into existing payment gateways and core banking systems.
- **Explainability:** each alert should show the main reasons for the high score so that analysts can take a quick and justified decision.
- **Auditability:** every score, decision and analyst action must be logged for audits and dispute handling.
- **Easy maintenance:** modular services and scripted model retraining so that rules and models can be updated without downtime.

Cost Analysis

- The proposed technology stack (Python, FastAPI, scikit-learn/XGBoost, PostgreSQL, Redis and React) is open source, so there is no licence cost for the software itself; the main costs are hosting, storage, development effort and later maintenance.
- Commercial fraud-prevention products are usually priced per transaction or by subscription, which becomes expensive at high volumes. A custom system can be cheaper in the long run and can be tuned to an organisation's own data.
- The cost has to be weighed against the loss it prevents: direct fraud losses, chargeback and investigation fees, and the customer churn caused by both fraud and wrongly declined genuine payments.
- Automation lowers the manual review effort, because analysts only see the small fraction of transactions that the model marks as risky.

Limitation

- **Data dependence:** model quality depends on the quantity and quality of labelled data; confirmed fraud labels arrive late (after disputes/chargebacks).
- **Trade-off between false positives and missed fraud:** no threshold removes both; every setting accepts some errors.
- **Adversarial behaviour:** fraudsters adapt, so detection needs constant monitoring and retraining.
- **Explainability of complex models:** ensemble and deep models are harder to explain than simple rules.
- **Security concerns:** the system holds sensitive financial information, so a breach would be very costly; strong security controls are mandatory.

Future Scope

- Deep-learning and graph-based models that detect fraud rings by linking shared devices, addresses and merchants.
- Unsupervised and semi-supervised anomaly detection to spot completely new fraud patterns without labels.
- Behavioural biometrics and device fingerprinting as additional risk signals.
- Explainable AI techniques (such as SHAP values) to show analysts why a transaction was flagged.
- Federated learning so that banks can improve a shared model without exchanging raw customer data.
- Extension to other payment channels such as UPI, wallets and net banking.

Experiment 2: Requirement Analysis and Software Requirement Specification (SRS)

Aim:- To do requirement analysis and develop the Software Requirement Specification sheet (SRS) for the Credit Card Fraud Detection System, following the IEEE SRS structure.

1) Introduction

1.1) Purpose

This Software Requirements Specification (SRS) gives a detailed description of the Credit Card Fraud Detection System to be developed. It states the functional and non-functional requirements, the external interfaces and the constraints of the system, and it is intended for developers, testers, project guides and the bank's fraud-operations staff. The primary purposes of the system are:

- **A) Detecting fraudulent transactions:** scoring every card transaction in real time and classifying it as genuine, suspicious or fraudulent.
- **B) Supporting fraud operations:** assisting analysts in reviewing alerts, taking decisions and blocking compromised cards quickly.
- **C) Reducing losses and false alarms:** protecting cardholders and the issuing bank while keeping wrongly declined genuine transactions to a minimum.

1.2) Scope

The scope of the Credit Card Fraud Detection System includes:

- **A) Transaction Ingestion:** receiving transaction requests from a payment gateway through a secure API and validating them.
- **B) Feature Engineering:** building behavioural features from the transaction and the card's recent history.
- **C) Fraud Scoring:** combining a machine-learning model and a rule engine to produce a fraud risk score.
- **D) Decision and Alerting:** approving, holding or declining a transaction and creating alerts with cardholder notifications.
- **E) Case Management:** letting analysts review alerts, record verdicts and block or unblock cards.
- **F) Model Management:** training, evaluating and deploying new model versions from confirmed labels.
- **G) Admin Panel and Reports:** user and role management, rule/threshold configuration and performance reports.

The system does not perform the actual payment authorisation, clearing or settlement; it only advises the authorisation flow with a risk decision.

1.3) Definitions, Acronyms, Abbreviations

Term	Meaning
Fraud	Unauthorised use of a card or card details

Term	Meaning
PAN	Primary Account Number (the card number); stored only in tokenised/masked form
ML	Machine Learning
Risk score	A value between 0 and 1 expressing the probability that a transaction is fraudulent
False positive	A genuine transaction wrongly flagged as fraud
False negative	A fraudulent transaction that is missed
Chargeback	Reversal of a transaction disputed by the cardholder
Precision / Recall	Share of flagged transactions that are truly fraud / share of all fraud that is caught
PCI DSS	Payment Card Industry Data Security Standard
API	Application Programming Interface
REST	Representational State Transfer, a web API style using HTTP and JSON
RBAC	Role-Based Access Control
GUI / DBMS / SQL	Graphical User Interface / Database Management System / Structured Query Language
OTP	One-Time Password used for step-up verification

1.4) Overview

Functional overview:

- Real-time scoring of incoming card transactions received from the payment gateway.
- Hybrid detection using an ML model and configurable rules.
- Alert generation, cardholder notification and analyst review workflow.
- Model retraining from confirmed fraud / false-positive labels.
- Admin panel for users, rules, thresholds and reports.

User experience:

- Clean analyst dashboard showing the alert queue, risk score and the top reasons for each alert.
- Secure handling of all card and customer data.
- Near real-time updates of alert status and key fraud metrics.

2) Overall Description (Product Perspective)

- Fraud prevention: catches suspicious transactions before or immediately after authorisation, reducing financial loss.
- Better customer experience: fewer wrongly declined genuine payments and faster confirmation of suspicious ones.
- Data insights: the stored scores and labels reveal fraud trends, risky merchants and peak fraud hours for future rule and model improvements.

2.1) System Interfaces

- **API integration:** a REST/JSON API through which the payment gateway sends transactions and receives the decision; an SMS/e-mail provider API for notifications.
- **Database:** a relational database (PostgreSQL) for transactions, alerts, cases, users and model metadata, and an in-memory cache (Redis) for fast access to recent card activity.

2.1.2) User Interfaces

- **Analyst interface:** web dashboard for alert queue, transaction details, card history and verdict submission.
- **Admin interface:** screens for user management, rules and thresholds, model versions and reports.

2.1.3) Hardware Interfaces

- **Server hardware:** application, model-inference and database servers (physical or cloud virtual machines).
- **Client hardware:** analysts and administrators use standard desktop/laptop web browsers.

2.1.4) Communication Interfaces

- All communication with the payment gateway, the notification provider and the dashboard uses HTTPS (TLS 1.2 or higher).
- Internal services communicate over a private network using HTTP/gRPC; database connections use SSL.

2.1.5) Memory Constraints

- Recent-activity features are cached in memory with expiry, and the model is loaded once per inference worker, to keep memory usage predictable.

2.1.6) Operation

- The scoring service runs continuously (24x7). Model training and report generation run as scheduled offline jobs so they do not affect live scoring.

2.1.7) Site Adaptation Requirements

- The dashboard should work on current versions of Google Chrome, Mozilla Firefox, Microsoft Edge and Safari, and be responsive to different screen sizes.
- Thresholds, rules and notification templates must be configurable per deployment without code changes.

2.2) Product Functions

- Receive and validate transaction requests.
- Compute features and score transactions (ML + rules).
- Approve, hold or decline transactions and raise alerts.
- Notify cardholders and collect their confirmation.
- Analyst alert review, card block/unblock and verdict recording.
- Model retraining, evaluation and deployment.
- Admin panel and reporting.

2.3) User Characteristics

- **Fraud analysts:** trained in fraud investigation; need clear, fast tools to judge alerts.
- **Administrators:** technically skilled staff who manage users, rules, thresholds and models.
- **Cardholders:** general public with varying technical skill; they only interact through SMS/e-mail confirmation.
- **Payment gateway:** an external system that interacts only through the API.

2.4) Constraints

- **Performance:** scoring must be fast enough for the authorisation flow, even at peak load.
- **Regulatory:** must follow PCI DSS guidance and applicable data-protection rules; full card numbers must not be stored in the clear.
- **Data:** confirmed fraud labels are few and arrive late, so the training data is highly imbalanced.
- **Integration:** the gateway and notification provider interfaces are controlled by third parties.

2.5) Assumptions and Dependencies

- a) A reliable network connection exists between the payment gateway and the system.
- b) Enough labelled historical data (or a public benchmark dataset) is available to train the initial model.
- c) The system depends on APIs of the payment gateway and the SMS/e-mail provider.
- d) Analysts have basic computer literacy and are trained on the alert workflow.

3) Specific Requirements

3.1) External Interfaces

- Payment Gateway API: receives a transaction request and returns the risk score and decision (APPROVE / HOLD / DECLINE).
- Notification API (SMS/e-mail): sends alerts and receives delivery status.
- Dashboard API: secured REST endpoints used by the web dashboard.

3.2) Functional Requirements

ID	Requirement	Priority
FR-1	The system shall accept a transaction request (card token, amount, currency, merchant, time, location, channel) through a secure API and validate its format.	High
FR-2	The system shall compute behavioural features from the transaction and the card's recent history.	High
FR-3	The system shall produce a fraud risk score between 0 and 1 using the active ML model and rule engine.	High
FR-4	The system shall return APPROVE, HOLD or DECLINE based on configurable score thresholds.	High
FR-5	The system shall create a fraud alert for every HOLD or DECLINE decision.	High

ID	Requirement	Priority
FR-6	The system shall notify the cardholder of suspicious transactions and record the reply.	Medium
FR-7	Analysts shall be able to view, filter, assign and review alerts with the transaction and card history.	High
FR-8	Analysts shall be able to mark an alert as confirmed fraud or false positive and block/unblock the card.	High
FR-9	Administrators shall be able to manage users and roles.	Medium
FR-10	Administrators shall be able to configure rules and thresholds.	Medium
FR-11	The system shall store confirmed labels and allow administrators to retrain, evaluate and deploy a new model version.	Medium
FR-12	The system shall generate reports on fraud rate, detection rate, false-positive rate and alert backlog.	Medium
FR-13	The system shall log every score, decision and analyst action for audit.	High

3.3) Performance Requirements

- **a) Response time:** the scoring API should return a decision within about 200 ms for 95% of requests (target).
- **b) Load handling:** the system should handle hundreds of transactions per second and scale horizontally during peak periods.
- **c) Model quality:** the model should be evaluated on precision, recall, F1-score and PR-AUC on a held-out test set before deployment.

3.4) Logical Database Requirements

- **a) Data model:** entities Customer, Card, Merchant, Transaction, ML_Model, Fraud_Alert and Analyst with the relationships shown in the ER diagram (Experiment 4).
- **b) Data integrity:** primary/foreign key constraints, non-null checks on mandatory fields and unique card tokens.
- **c) Retention:** transaction and audit data retained for the period required by the organisation's policy.

3.5) Design Constraints

- Backend in Python (FastAPI); ML with scikit-learn / XGBoost; PostgreSQL database; Redis cache; React web dashboard.
- Services packaged as containers so that they can be deployed on any server or cloud platform.

3.6) Software System Attributes

- **a) Reliability:** no transaction request should be lost; if the ML service fails, the system falls back to the rule engine.
- **b) Availability:** the scoring service should be highly available, with health checks and redundancy.

- **c) Usability:** the dashboard should be simple and let analysts decide an alert in a few clicks.
- **d) Maintainability and scalability:** modular services, versioned models and configuration-driven rules.

3.7) Other Requirements

- **Data protection:** card numbers tokenised or masked; sensitive data encrypted in transit and at rest.
- **Authentication and authorization:** secure login, token-based sessions and role-based access, so users can only reach permitted functions.
- **Auditability:** tamper-evident logs of decisions and analyst actions.
- **Third-party software:** the system may integrate with external SMS/e-mail and identity services.

Experiment 3: Function-Oriented Diagrams: Data Flow Diagram (DFD) and Structure Chart

Aim:- To perform the function-oriented analysis of the Credit Card Fraud Detection System using Data Flow Diagrams (Level 0, Level 1, Level 2) and a Structure Chart.

Software Used:- StarUML / any UML-DFD drawing tool

Theory

Data Flow Diagrams (DFD) represent the flow of information through a system graphically instead of describing it in text. A DFD shows external entities (sources and destinations of data), processes (which transform data), data stores (where data rests) and data flows (labelled arrows). DFDs are drawn at successively deeper levels:

- **Level 0 DFD (context diagram):** shows the whole system as a single process together with its external entities and the data exchanged with them.
- **Level 1 DFD:** breaks the single process into its major sub-processes and shows the data stores they use.
- **Level 2 DFD:** explodes one Level 1 process into more detailed sub-processes, with inputs, processing and outputs.

A Structure Chart is the function-oriented design counterpart of a DFD: it shows the module hierarchy of the software, that is, which module calls which lower-level module.

Level 0 DFD (Context Diagram)

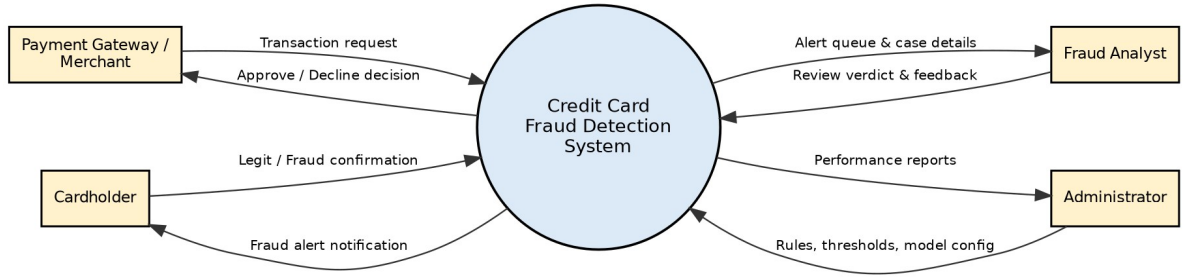


Fig. 3.1: Level 0 DFD of the Credit Card Fraud Detection System

The four external entities are the Payment Gateway/Merchant (sends transactions, receives the decision), the Cardholder (receives alerts and confirms transactions), the Fraud Analyst (reviews alerts) and the Administrator (configures rules, models and reads reports).

Level 1 DFD

The Level 1 DFD is split into two figures for readability. Figure 3.2 shows the real-time detection path (processes 1.0 to 4.0) and Figure 3.3 shows case review, model training and administration (processes 5.0 to 7.0).

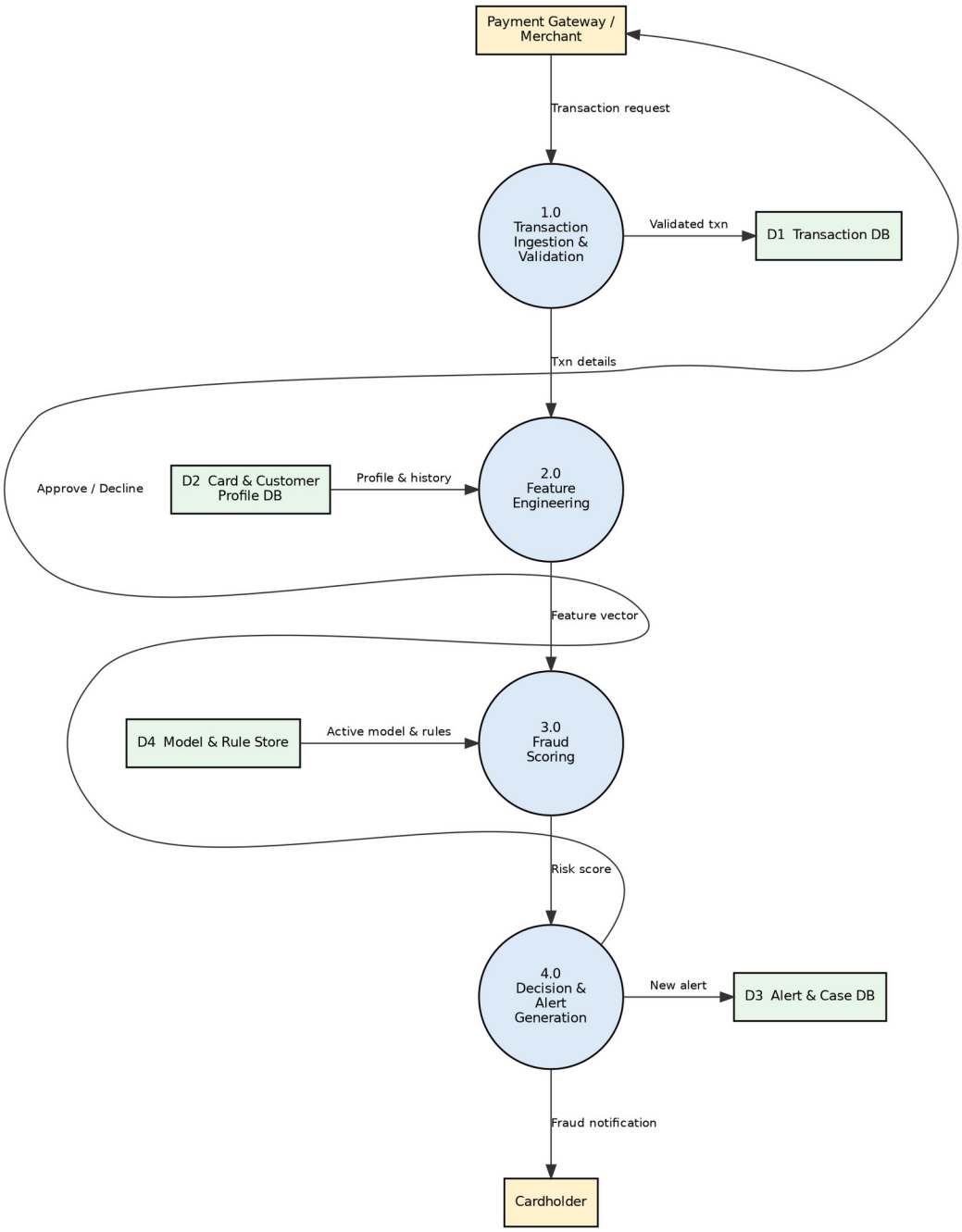


Fig. 3.2: Level 1 DFD (a): real-time detection path

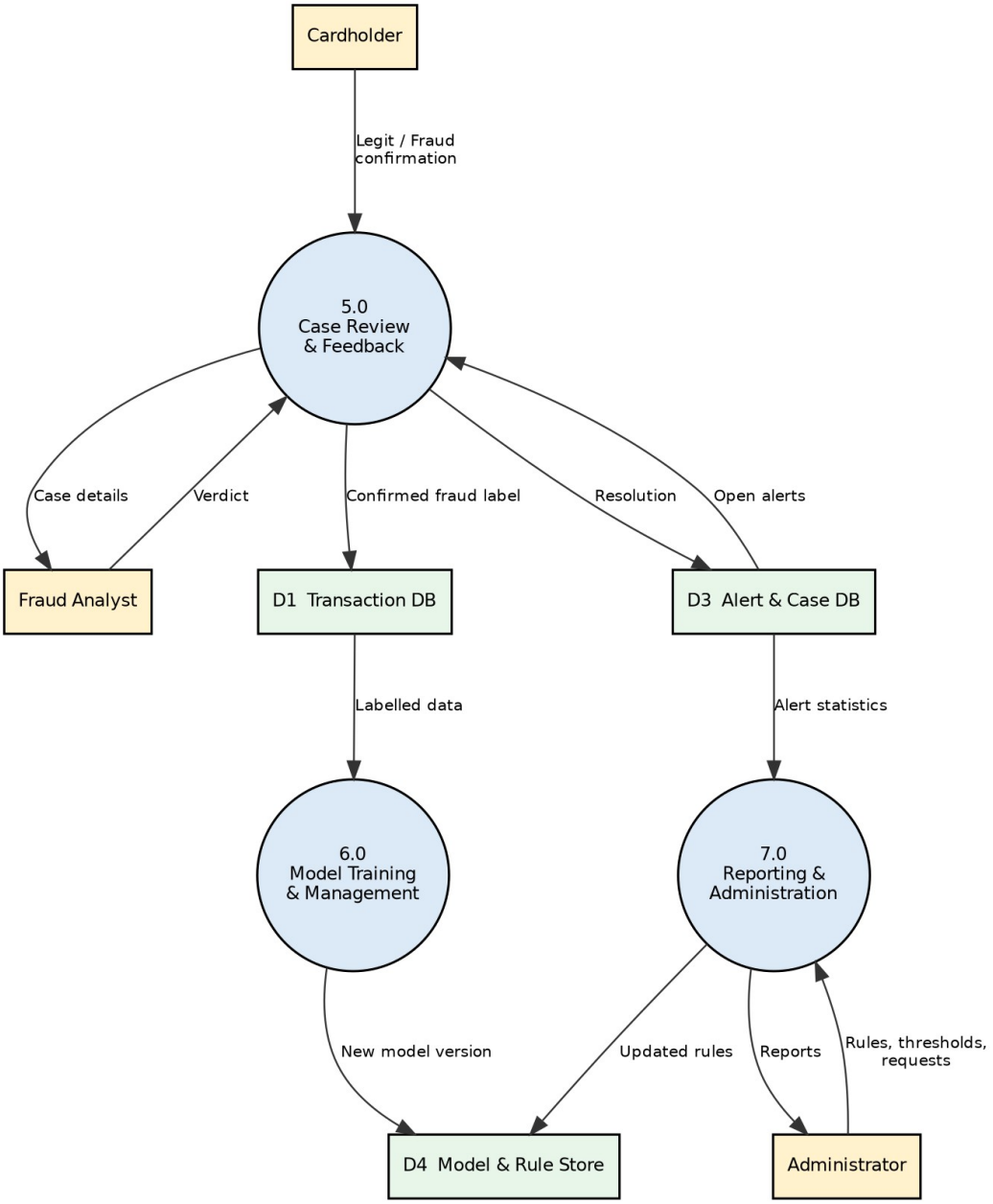


Fig. 3.3: Level 1 DFD (b): review, training and administration

Process	Description
1.0 Transaction Ingestion & Validation	Receives the request from the gateway, checks the format and card status, and stores the validated transaction in D1.
2.0 Feature Engineering	Reads the card and customer profile (D2) and recent history and builds the feature vector.
3.0 Fraud Scoring	Uses the active model and rules from D4 to compute the fraud risk score.
4.0 Decision & Alert Generation	Compares the score with thresholds, returns APPROVE / HOLD / DECLINE, creates alerts (D3) and notifies the cardholder.
5.0 Case Review & Feedback	Shows open alerts to analysts, records verdicts and cardholder confirmations and stores confirmed labels.

Process	Description
6.0 Model Training & Management	Retrains the model on labelled data from D1 and stores the new version in D4.
7.0 Reporting & Administration	Handles rule/threshold configuration, user requests and generates reports from stored data.

Level 2 DFD (Explosion of Process 3.0: Fraud Scoring)

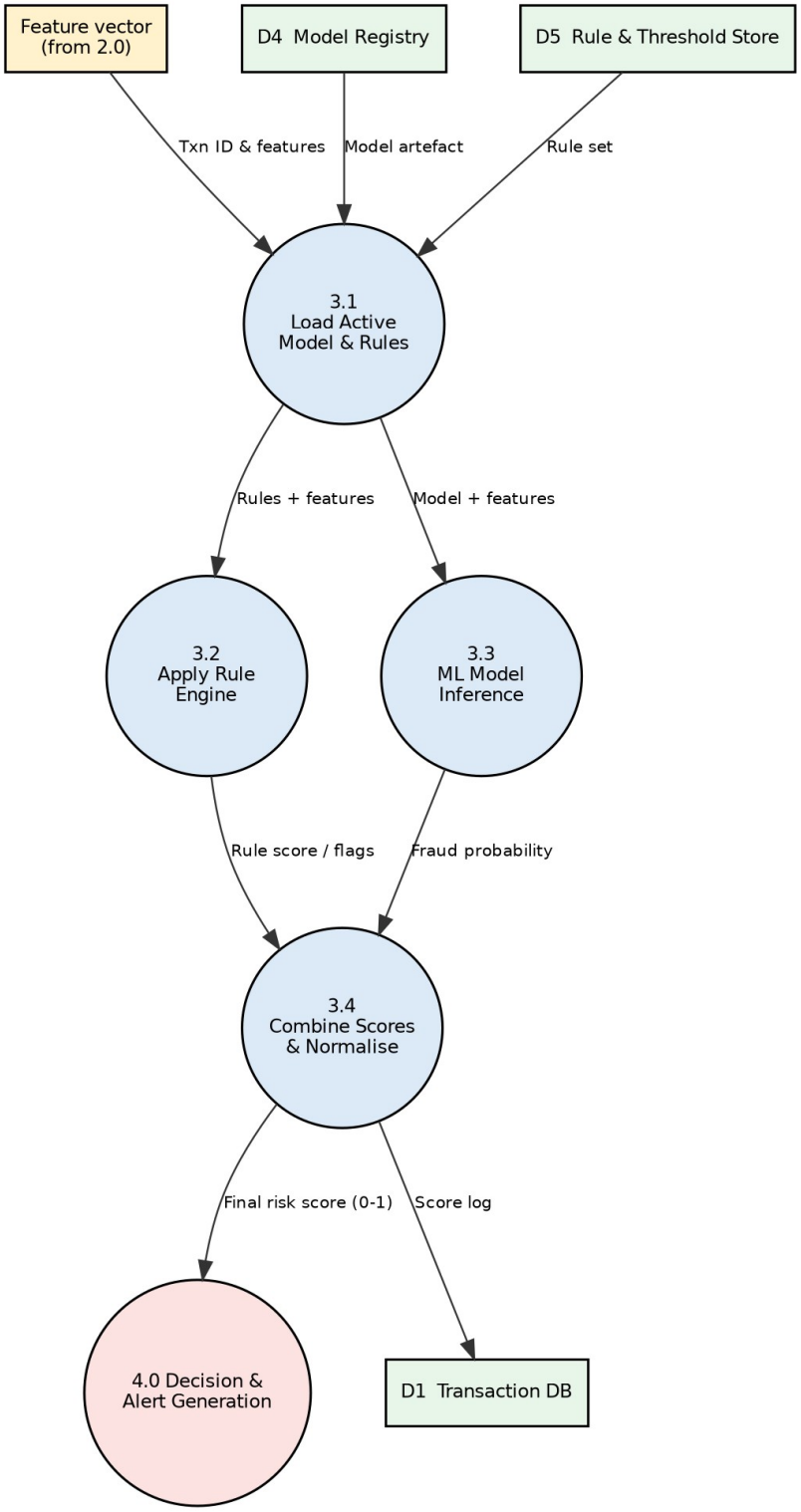


Fig. 3.4: Level 2 DFD of Process 3.0 Fraud Scoring

Process 3.0 is split into loading the active model and rule set (3.1), applying the rule engine (3.2), running the ML model inference (3.3) and combining both outputs into one normalised risk score (3.4), which is passed to Process 4.0 and logged in the transaction database.

Structure Chart

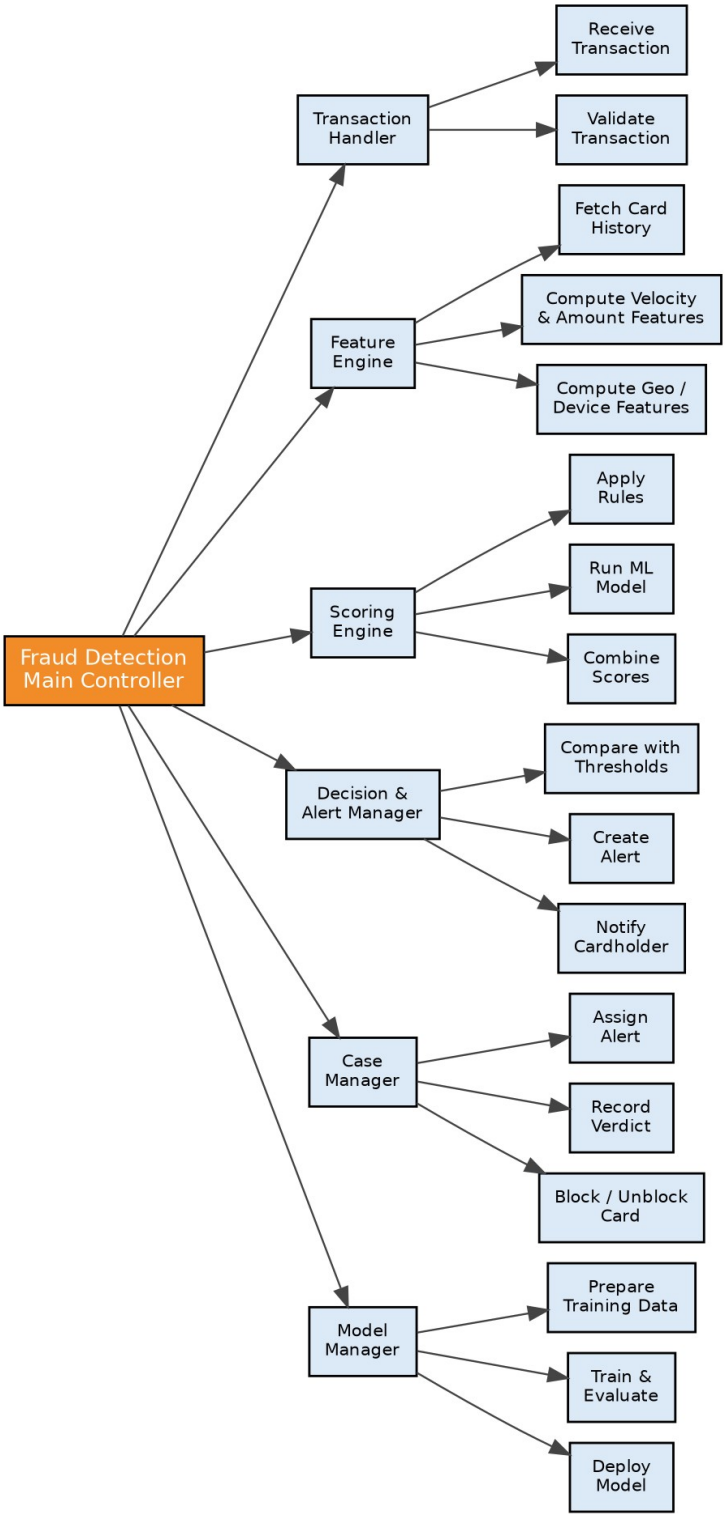


Fig. 3.5: Structure chart of the Credit Card Fraud Detection System

The Main Controller calls six first-level modules (Transaction Handler, Feature Engine, Scoring Engine, Decision & Alert Manager, Case Manager and Model Manager). Each of these calls its own lower-level modules, giving a modular, top-down design in which each module has one clear responsibility (high cohesion) and modules interact through simple data passed between them (low coupling).

Result:- The function-oriented view of the system was analysed and the DFDs and structure chart were drawn.

Experiment 4: Entity Relationship (ER) Diagram

Aim:- To draw the ER diagram for the Credit Card Fraud Detection System.

Theory

An ER diagram is composed of entities, relationships, attributes and cardinality. Entities are the things of interest about which data is stored, relationships are the associations between entities, attributes describe the entities, and cardinality states how many instances of one entity can be related to instances of another.

ER diagram components and features

- **Entities:** things of interest, such as Customer, Card or Transaction.
- **Relationships:** connections between entities, such as a Card making Transactions.
- **Attributes:** descriptors of entities, such as a transaction's amount.
- **Cardinality:** how many entities can be related (1:1, 1:N, M:N).
- **Entity categories:** strong, weak and associative entities.
- **Entity keys:** super, candidate and primary keys (PK); a foreign key (FK) links to the primary key of another entity.
- **Attribute categories:** simple, composite, derived, single-valued and multi-valued attributes.

ERD for the Credit Card Fraud Detection System

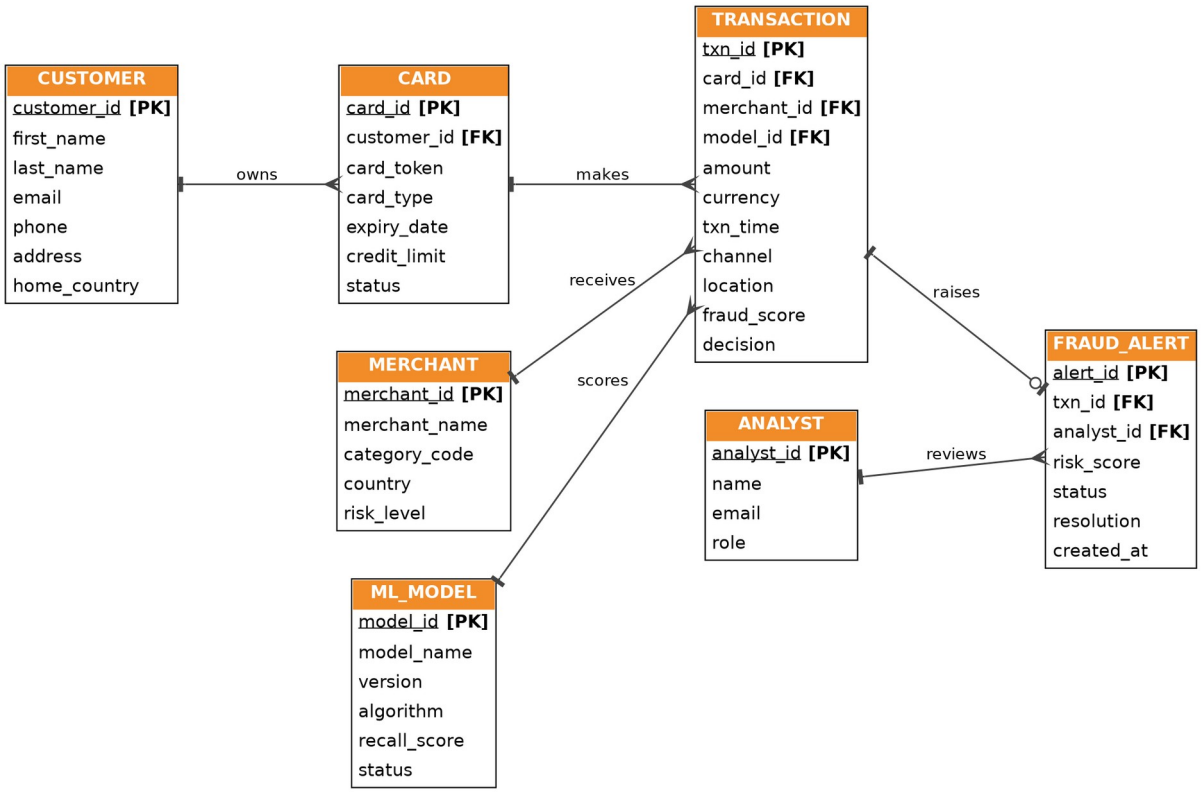


Fig. 4.1: ER diagram (crow's-foot notation)

The ER diagram shows the seven entities of the system and the relationships between them. Underlined attributes marked [PK] are primary keys and attributes marked [FK] are foreign keys.

Entities and their Attributes

- **Customer:** customer_id (primary key), first_name, last_name, email, phone, address, home_country. Name is a composite attribute made of first and last name.
- **Card:** card_id (primary key), customer_id (foreign key), card_token, card_type, expiry_date, credit_limit, status. Only a token/masked value of the card number is stored.
- **Merchant:** merchant_id (primary key), merchant_name, category_code, country, risk_level.
- **Transaction:** txn_id (primary key), card_id, merchant_id and model_id (foreign keys), amount, currency, txn_time, channel (POS/online), location, fraud_score, decision.
- **ML_Model:** model_id (primary key), model_name, version, algorithm, recall_score, status (active/retired).
- **Fraud_Alert:** alert_id (primary key), txn_id and analyst_id (foreign keys), risk_score, status, resolution (confirmed fraud / false positive), created_at.
- **Analyst:** analyst_id (primary key), name, email, role.

Relationships between Entities

Relationship	Entities	Cardinality	Explanation
owns	Customer - Card	1 : N	A customer can own many cards, but each card belongs to one customer.
makes	Card - Transaction	1 : N	A card can be used for many transactions; each transaction uses one card.
receives	Merchant - Transaction	1 : N	A merchant receives many transactions; each transaction is made at one merchant.
scores	ML_Model - Transaction	1 : N	One model version scores many transactions; each transaction is scored by one model version.
raises	Transaction - Fraud_Alert	1 : 0..1	A transaction may raise at most one alert; an alert belongs to exactly one transaction.
reviews	Analyst - Fraud_Alert	1 : N	An analyst reviews many alerts; each alert is assigned to one analyst.

Result:- The ER diagram of the Credit Card Fraud Detection System was drawn with its entities, keys and cardinalities.

Experiment 5: User's View Analysis: Use Case Diagram

Aim:- To perform the user's view analysis for the Credit Card Fraud Detection System using a use case diagram.

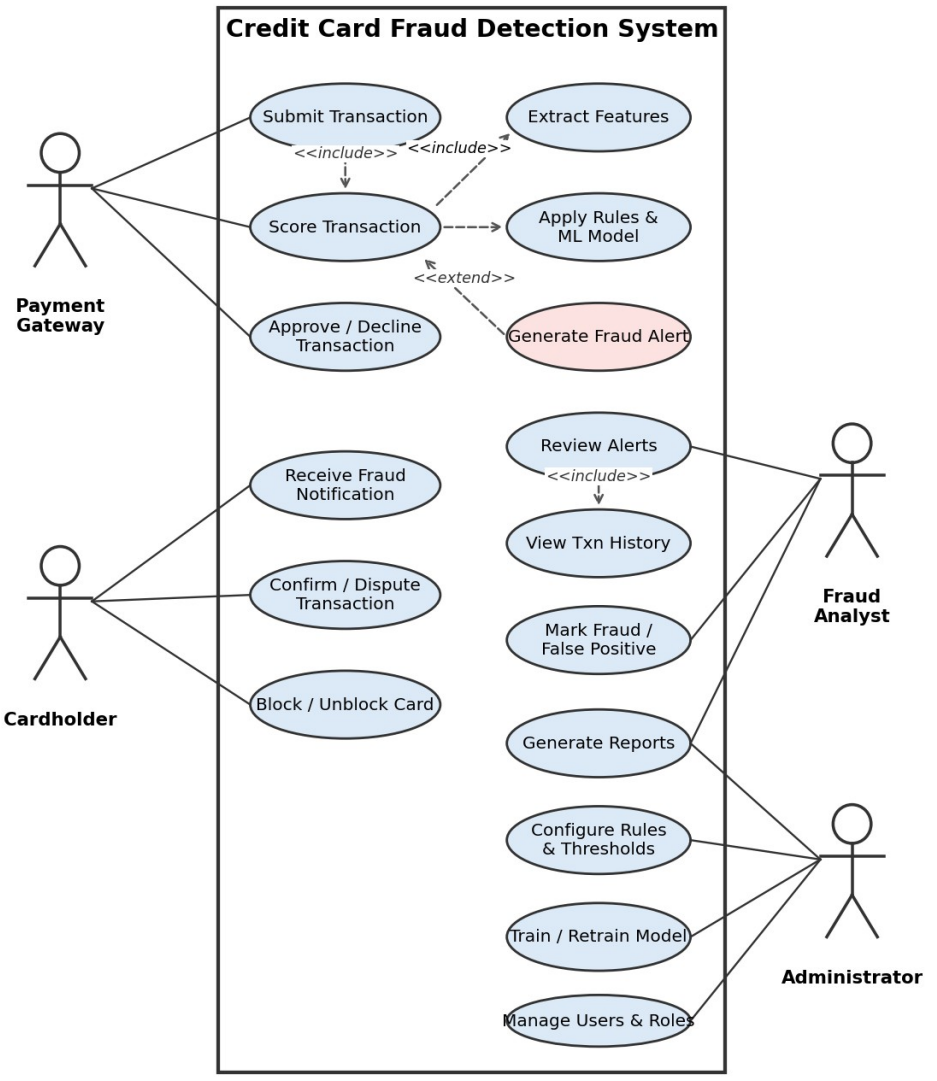
Software Used:- StarUML

Theory

Use case diagrams are behavioural diagrams that show the interactions between a system and its users (actors). They are used to capture the functional requirements of a system and are mainly used in the analysis and design phases. The components of a use case diagram are:

- **Actor:** a person, organisation or external system that interacts with the system to achieve a goal (drawn as a stick figure).
- **System boundary:** the rectangle that encloses the use cases of the system being modelled.
- **Use case:** a sequence of interactions between an actor and the system that gives the actor a useful result (drawn as an ellipse).
- **Communication link / association:** a line joining an actor to a use case it takes part in.
- **Include:** a dashed arrow labelled <<include>> from a use case to another use case whose behaviour is always used by it.
- **Extend:** a dashed arrow labelled <<extend>> from an optional use case to the base use case it extends under a condition.
- **Generalization:** a solid line with a hollow triangle showing that an actor or use case is a specialised form of another.

Use Case Diagram



Dashed arrows: <<include>> points to the included use case; <<extend>> points to the base use case being extended.

Fig. 5.1: Use case diagram of the Credit Card Fraud Detection System

Actors

Actor	Description
Payment Gateway (external system)	Sends card transactions for scoring and receives the decision.
Cardholder	Receives fraud notifications, confirms or disputes transactions and can block a card.
Fraud Analyst	Reviews alerts, studies transaction history, marks verdicts and generates reports.
Administrator	Manages users, rules, thresholds and models, and generates reports.

Use Case Descriptions

UC-01: Score Transaction

Item	Description
Primary actor	Payment Gateway
Precondition	Transaction request is received through the API and the gateway is authenticated.
Main flow	1. System validates the request. 2. System extracts features (included use case). 3. System applies rules and the ML model (included use case). 4. System computes the risk score and returns the decision.
Extension	If the score is above the alert threshold, the use case Generate Fraud Alert is executed.
Postcondition	The transaction, score and decision are stored in the database.

UC-02: Review Alerts

Item	Description
Primary actor	Fraud Analyst
Precondition	Analyst is logged in and at least one alert is open.
Main flow	1. Analyst opens the alert queue. 2. Analyst selects an alert and views details and card history (included use case). 3. Analyst marks the alert as confirmed fraud or false positive. 4. System saves the verdict and blocks or releases as needed.
Postcondition	Alert is closed and the label is stored for model retraining.

UC-03: Confirm / Dispute Transaction

Item	Description
Primary actor	Cardholder
Precondition	A suspicious-transaction notification was sent to the cardholder.
Main flow	1. Cardholder receives the SMS/e-mail. 2. Cardholder replies that the transaction is genuine or not genuine. 3. System records the reply and updates the alert.
Postcondition	The alert is updated and, if not genuine, the card can be blocked.

UC-04: Train / Retrain Model

Item	Description
Primary actor	Administrator
Precondition	Administrator is logged in and new labelled data is available.

Item	Description
Main flow	1. Administrator starts retraining. 2. System trains and evaluates the new model (precision, recall, F1, PR-AUC). 3. Administrator compares it with the active model and approves deployment.
Postcondition	The new model version becomes the active model in the model registry.

Result:- The use case diagram with actors, use cases and their relationships was drawn for the system.

Experiment 6: Structural View: Class Diagram and Object Diagram

Aim:- To draw the structural view diagrams (class diagram and object diagram) for the Credit Card Fraud Detection System.

Software Used:- StarUML

Theory

A class diagram is a UML structural diagram that represents the structure of a system in terms of classes and interfaces. It shows the attributes and operations of classes and the relationships between them (association, generalization, dependency, etc.). Class diagrams map directly to object-oriented languages. An object diagram is a snapshot of the system at a particular moment: it shows the instances (objects) of classes with their attribute values and the links between them. Object diagrams can be seen as instance-level class diagrams and are useful for explaining small parts of a complex system.

Class Diagram

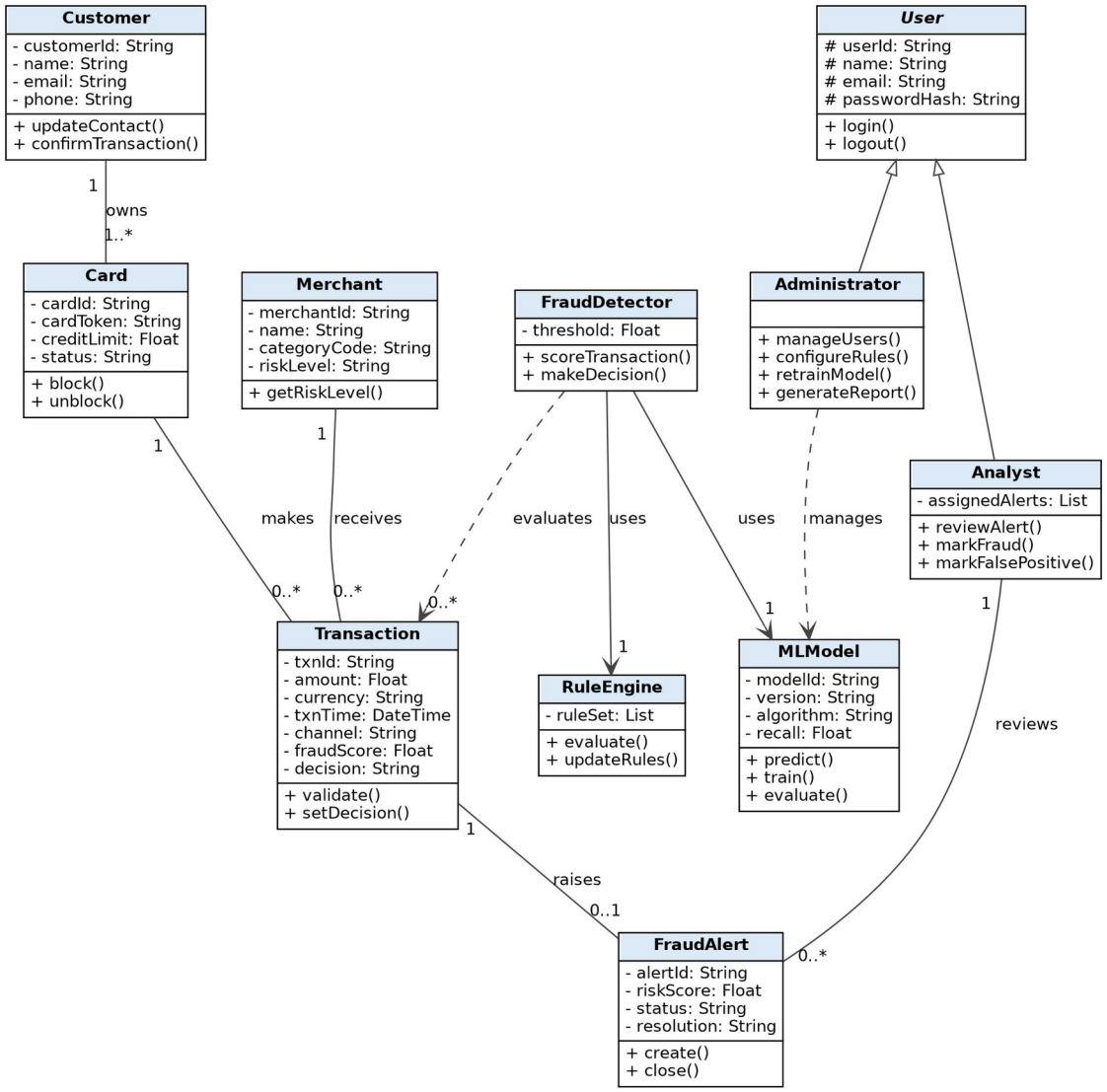


Fig. 6.1: Class diagram of the Credit Card Fraud Detection System

- **Customer, Card, Merchant, Transaction:** the core business classes. A customer owns 1..* cards; a card makes 0..* transactions; a merchant receives 0..* transactions.
- **FraudDetector:** the service class that evaluates transactions; it uses one RuleEngine and one MLModel to produce a score and decision.
- **FraudAlert:** raised by a transaction (0..1) and reviewed by an Analyst.
- **User (abstract):** parent class of Analyst and Administrator using generalization; it holds the common login data and operations. The Administrator manages the MLModel and the rules.

Object Diagram

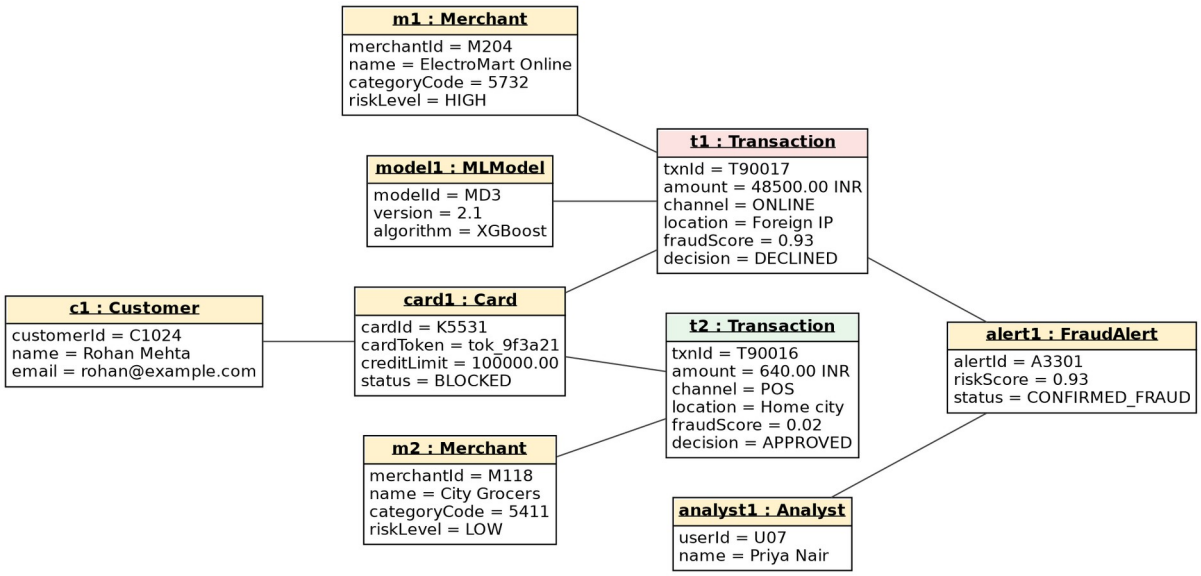


Fig. 6.2: Object diagram (snapshot of one fraud case)

The object diagram shows one snapshot: customer c1 owns card1 (now blocked), which made two transactions. Transaction t2 (a small POS purchase at a low-risk merchant) scored 0.02 and was approved. Transaction t1 (a large online purchase from a foreign location at a high-risk merchant) was scored 0.93 by model1 and declined, which raised alert1, reviewed by analyst1 and confirmed as fraud. (The names and values are sample data only.)

Result:- The class diagram and the object diagram of the system were drawn.

Experiment 7: Behavioral View: State-Chart Diagram and Activity Diagram

Aim:- To draw the state-chart diagram and the activity diagram for the Credit Card Fraud Detection System.

Software Used:- StarUML

Theory

A state-chart (state machine) diagram describes the different states an object goes through during its life and the events that cause it to move from one state to another. An activity diagram shows the flow of control from one activity to another; it is like a flowchart with support for decisions, parallel flows and swimlanes that show which actor or component performs each activity.

State-Chart Diagram: Life Cycle of a Transaction

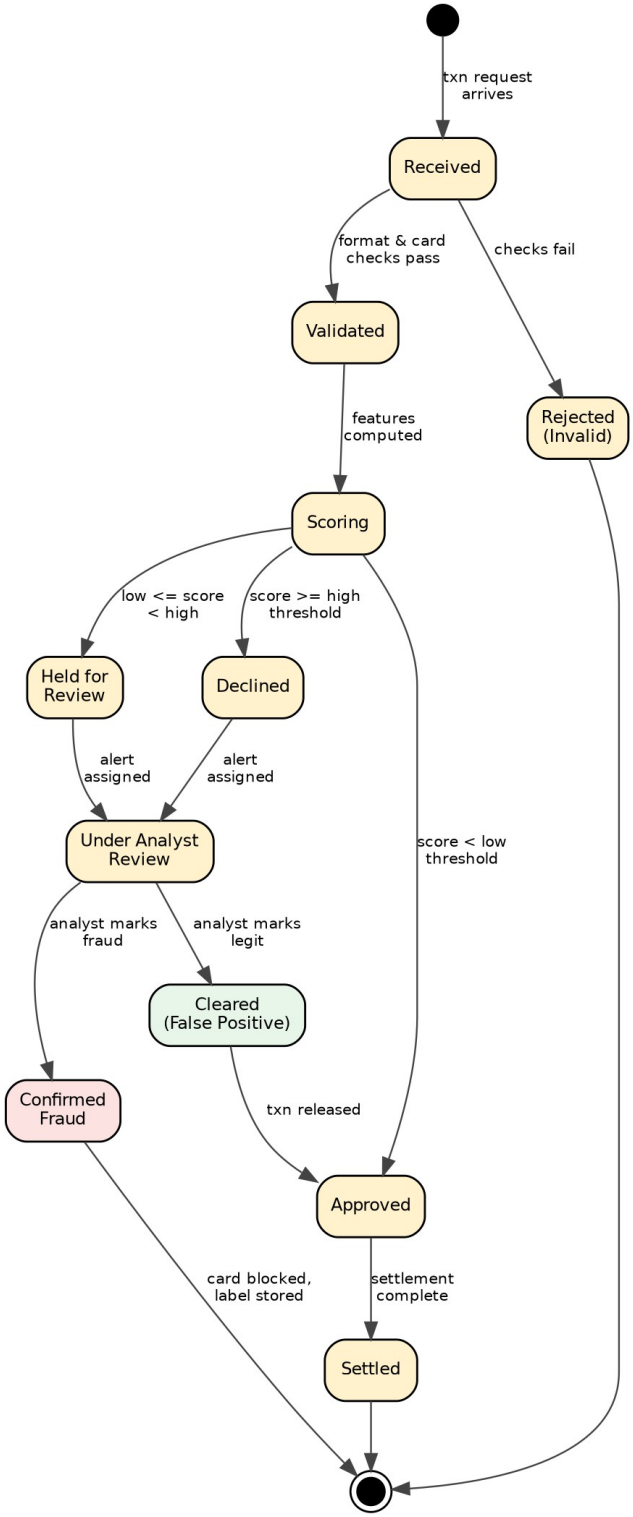


Fig. 7.1: State-chart diagram of a transaction

A transaction starts in the Received state. It is Rejected if the format or card checks fail; otherwise it is Validated and moves to Scoring once features are computed. Depending on the risk score it becomes Approved (low score), Held for Review (medium score) or Declined (high score). Held and declined transactions go to Under Analyst Review, where the analyst marks

them Confirmed Fraud (card blocked, label stored) or Cleared as a false positive (transaction released and approved). Approved transactions are finally Settled.

Activity Diagram (with Swimlanes)

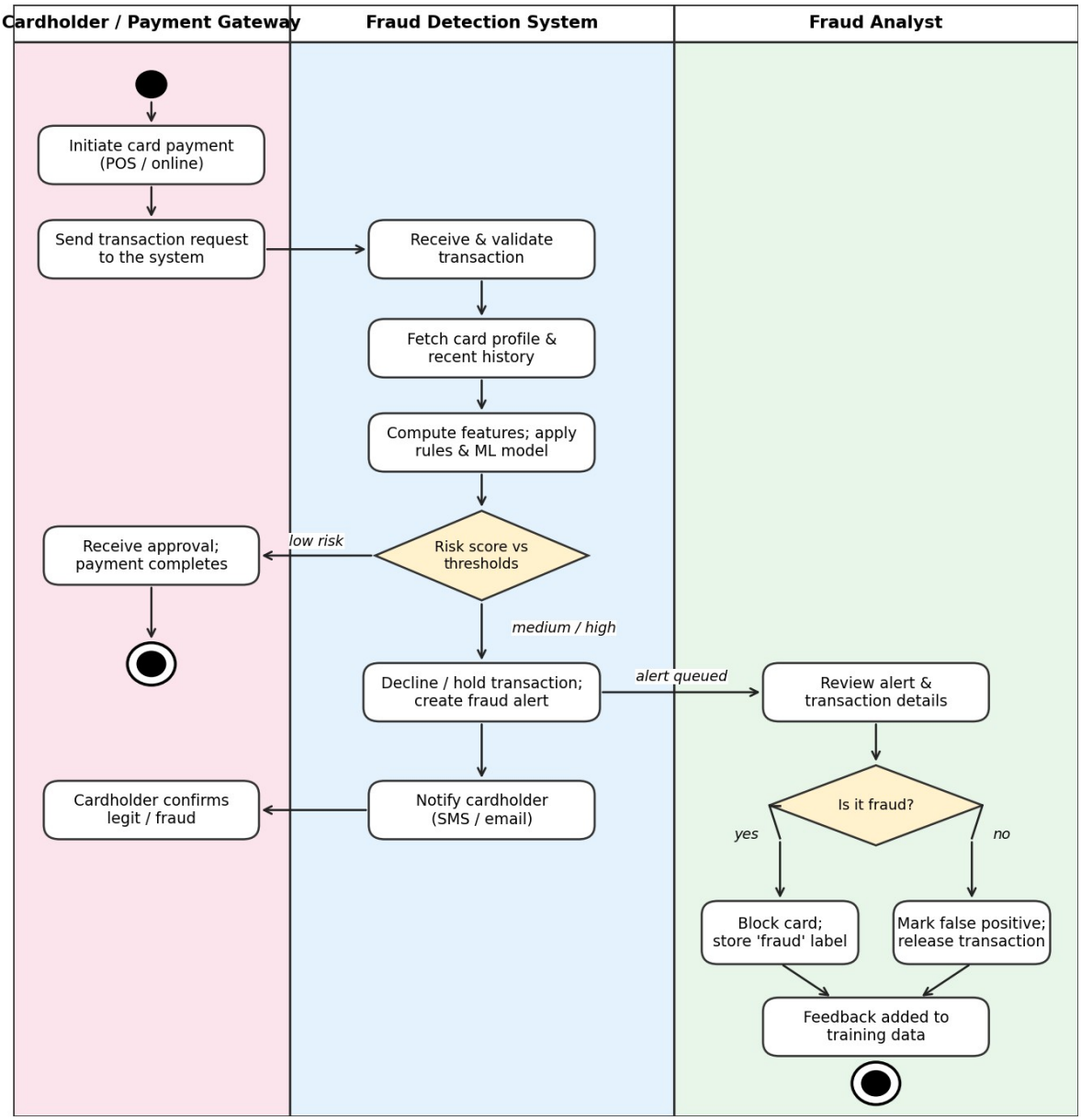


Fig. 7.2: Activity diagram of the fraud detection process

The swimlanes show the activities of the Cardholder/Payment Gateway, the Fraud Detection System and the Fraud Analyst. After a payment is initiated, the system validates it, fetches history, computes features and scores it. A low score leads to approval; a medium or high score leads to a decline/hold, an alert and a cardholder notification. The analyst reviews the alert and either blocks the card (fraud) or releases the transaction (false positive), and the outcome is added to the training data.

Result:- The state-chart diagram and the activity diagram were drawn for the system.

Experiment 8: Behavioral View: Sequence Diagram and Collaboration Diagram

Aim:- To perform the behavioral view diagrams (sequence diagram and collaboration diagram) for the Credit Card Fraud Detection System.

Software Used:- StarUML

Theory

Sequence diagram: A sequence diagram is the most commonly used interaction diagram. It depicts the interaction between objects in time order, that is, the order in which messages are exchanged. It is also called an event diagram or event scenario and shows how, and in what order, the objects of a system work together.

Collaboration diagram: A collaboration (communication) diagram shows the same information as a sequence diagram but emphasises the relationships between objects instead of time. The objects are drawn as nodes, linked to each other, and the messages between them are numbered to show their order.

Sequence Diagram for the Fraud Detection System

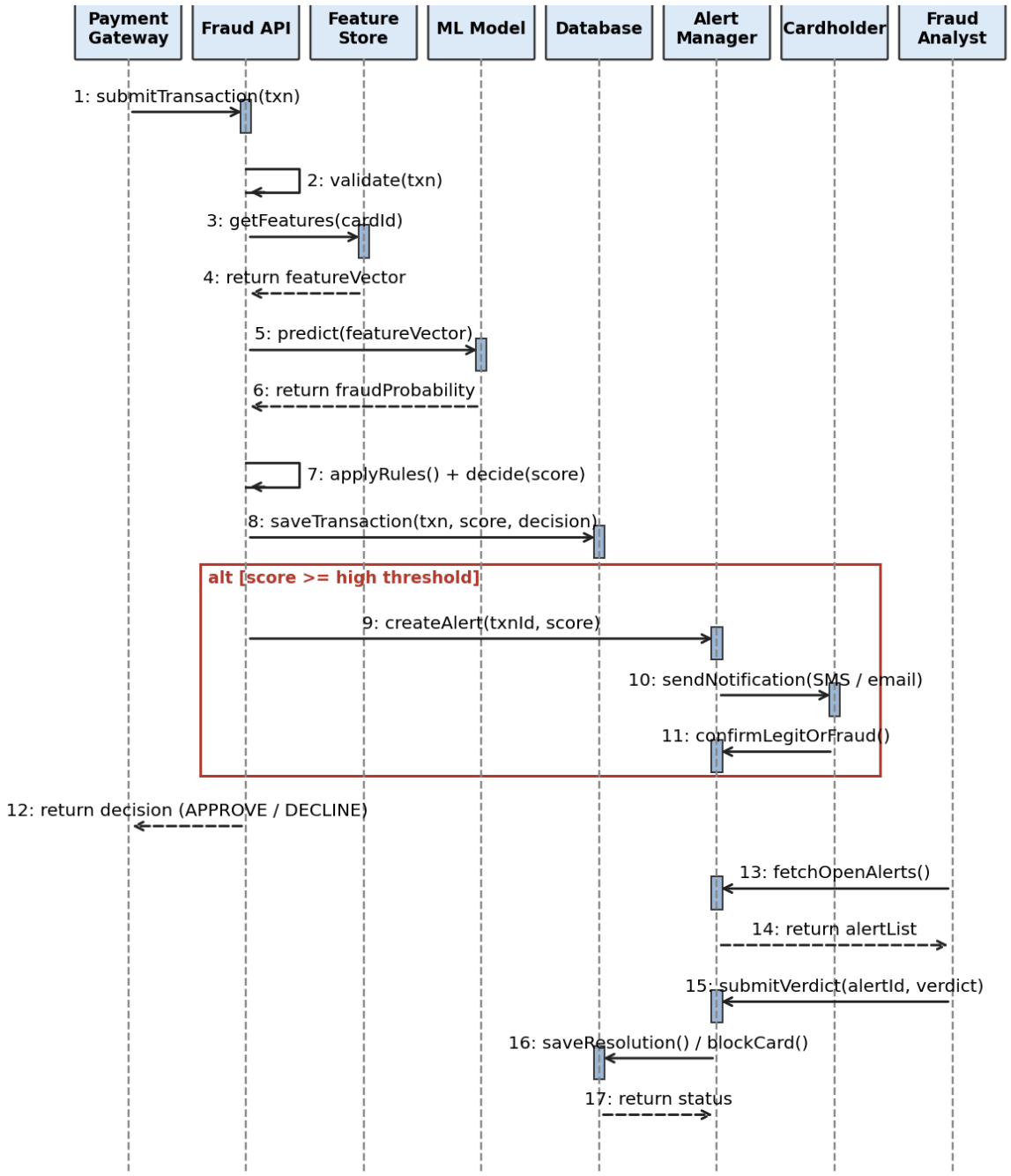


Fig. 8.1: Sequence diagram of transaction scoring and alert review

The gateway submits a transaction to the Fraud API, which validates it, obtains features from the Feature Store and the fraud probability from the ML Model, and applies the rules. The transaction and decision are saved in the database. In the alternative flow (score above the high threshold) an alert is created, the cardholder is notified and may confirm, and the DECLINE decision is returned to the gateway. Later, the analyst fetches the open alerts and submits a verdict, which the Alert Manager saves (and blocks the card if it is fraud).

Collaboration Diagram for the Fraud Detection System

Transaction scoring side

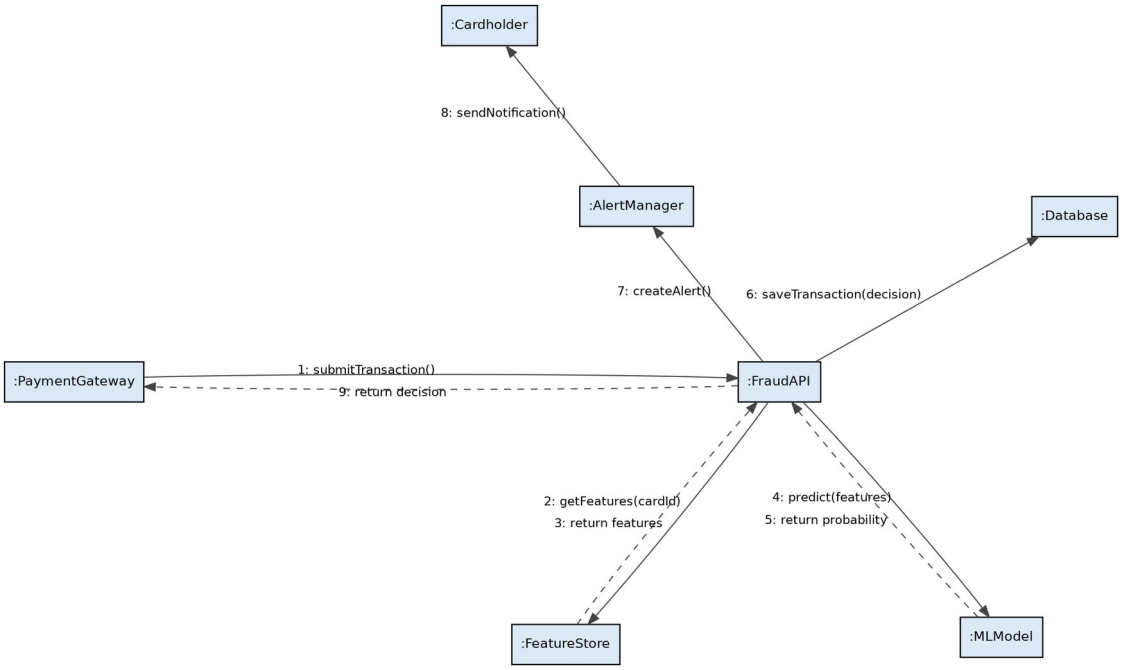


Fig. 8.2: Collaboration diagram: transaction scoring

Analyst side

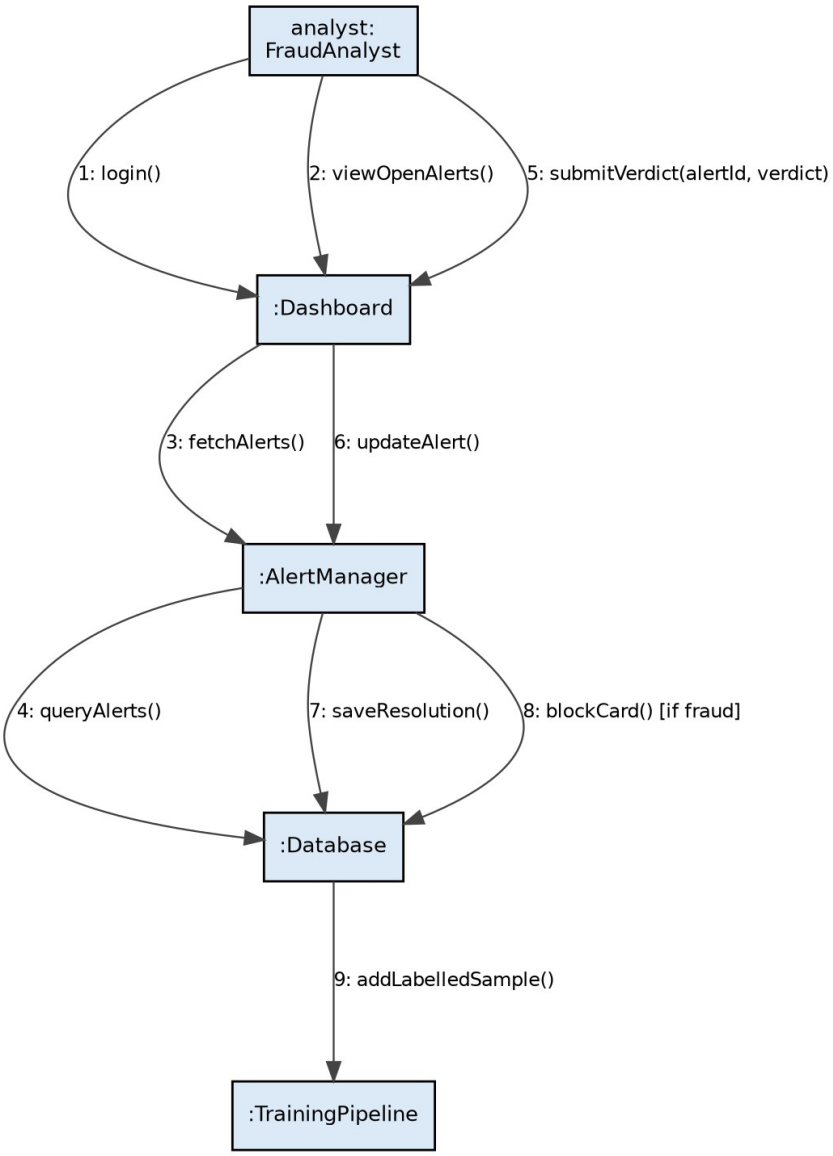


Fig. 8.3: Collaboration diagram: analyst alert review

Result:- The sequence diagram and the collaboration diagrams were drawn for the system.

Experiment 9 (CBS 1): Implementation View: Component Diagram

Aim:- To perform the implementation view diagram: component diagram for the Credit Card Fraud Detection System.

Software Used:- StarUML

Theory

A component diagram is used to break down a large object-oriented system into smaller components so that it becomes easier to manage. It models the physical view of a system, such as executables, files and libraries. It visualises the relationships and organisation between the components and helps in forming an executable system. A component is a single unit of the system that is replaceable and executable. Its implementation details are hidden and it needs an interface to execute a function; it is like a black box whose behaviour is described by its provided and required interfaces.

Component Diagram

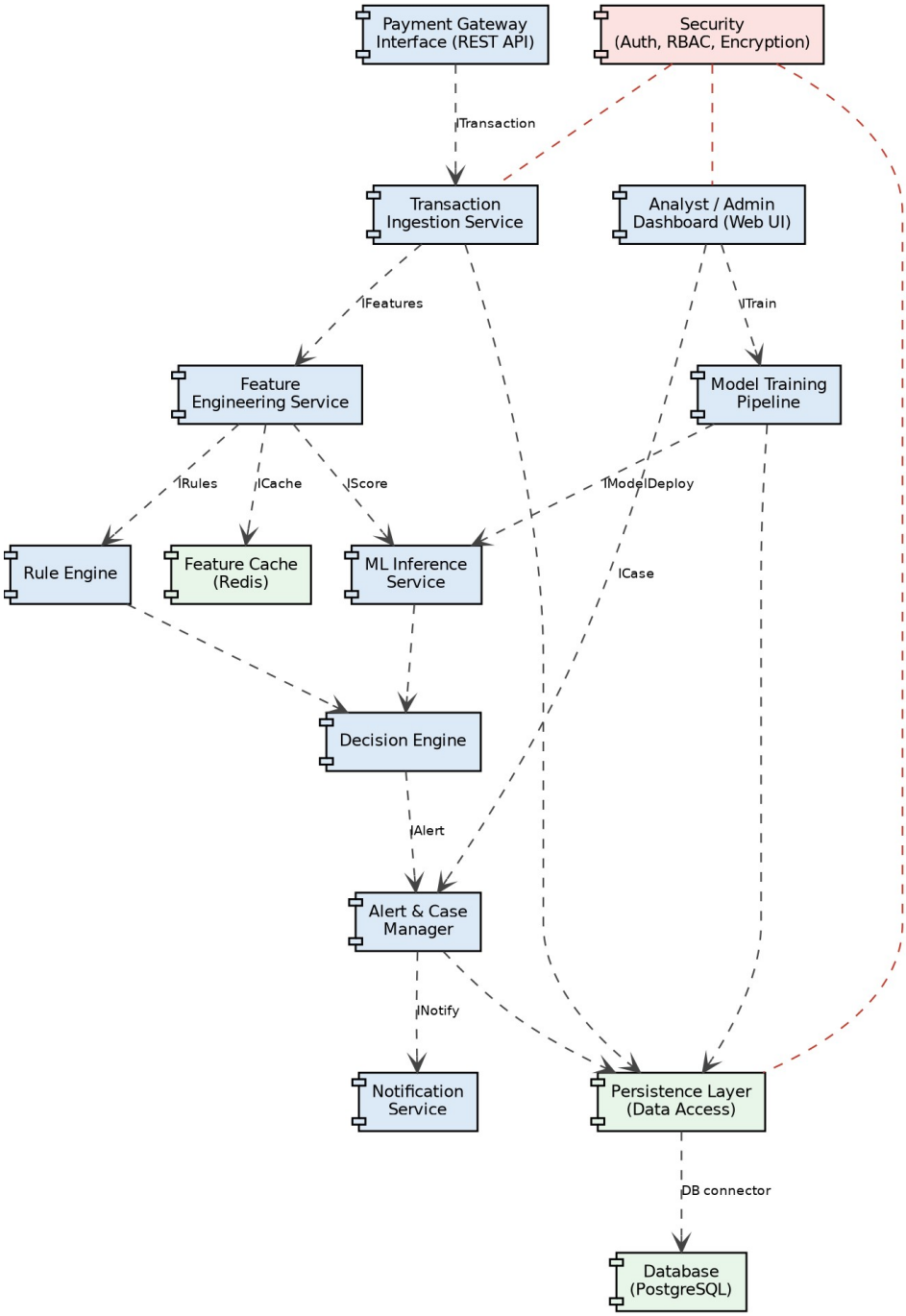


Fig. 9.1: Component diagram of the Credit Card Fraud Detection System

Component	Responsibility
Payment Gateway Interface	REST API through which the gateway submits transactions and gets decisions.
Transaction Ingestion Service	Validates and stores incoming transactions.
Feature Engineering Service	Builds behavioural features using the card history and the Redis feature cache.
Rule Engine	Applies configurable business rules and returns rule flags.

Component	Responsibility
ML Inference Service	Loads the active model and returns the fraud probability.
Decision Engine	Combines rule and model outputs and applies thresholds.
Alert & Case Manager	Creates alerts, assigns cases and records analyst verdicts.
Notification Service	Sends SMS/e-mail messages to cardholders.
Analyst / Admin Dashboard	Web UI for alert review, configuration and reports.
Model Training Pipeline	Retrains and evaluates models and deploys them to the inference service.
Security	Authentication, role-based access control and encryption used by other components.
Persistence Layer, Database, Feature Cache	Data-access layer with the PostgreSQL database (connected by a database connector) and the Redis cache.

Result:- The component diagram of the system was drawn.

Experiment 10 (CBS 2): Environmental View: Deployment Diagram

Aim:- To perform the environmental view diagram: deployment diagram for the Credit Card Fraud Detection System.

Software Used:- StarUML

Theory

The deployment diagram visualises the physical hardware on which the software will be deployed. It shows the static deployment view of a system, in terms of nodes and their relationships. It maps the software architecture created in design to the physical system architecture where the software is executed as nodes. Since many nodes are involved, the relationships between them are shown by communication paths.

Deployment Diagram

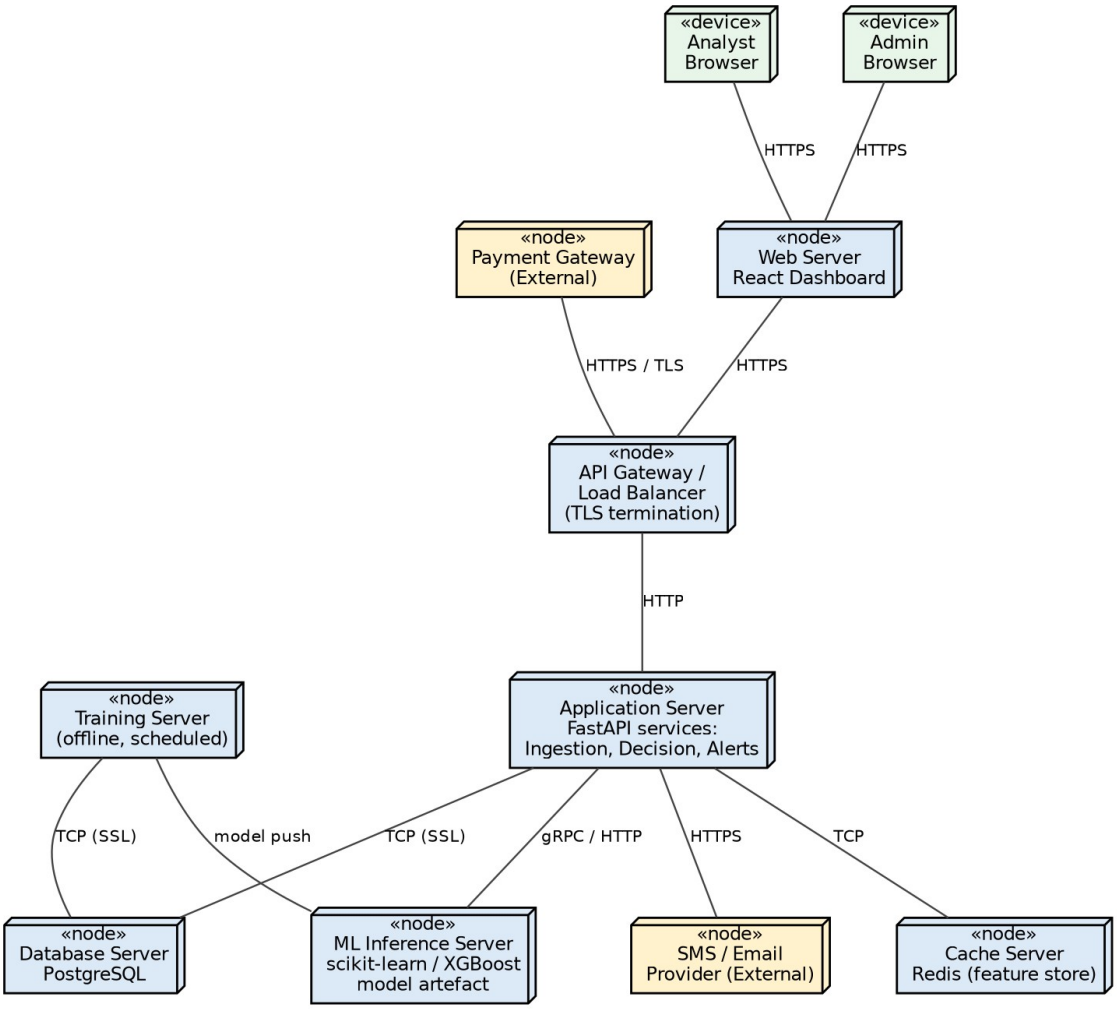


Fig. 10.1: Deployment diagram of the Credit Card Fraud Detection System

Node	Software deployed / role	Connection
Analyst / Admin Browser	Standard web browser used to access the dashboard	HTTPS to the web server
Web Server	Serves the React dashboard	HTTPS to the API gateway

Node	Software deployed / role	Connection
Payment Gateway (external)	Sends transaction requests	HTTPS/TLS to the API gateway
API Gateway / Load Balancer	TLS termination, authentication and request routing	HTTP to the application server
Application Server	FastAPI services: ingestion, decision engine, alert and case manager	gRPC/HTTP to the ML server; TCP to cache and database; HTTPS to SMS/e-mail provider
ML Inference Server	Trained model artefact (scikit-learn / XGBoost) served for real-time predictions	Receives model updates from the training server
Cache Server	Redis store for recent card activity used in feature computation	TCP
Database Server	PostgreSQL database for transactions, alerts, users and model metadata	TCP (SSL)
Training Server	Offline, scheduled model training and evaluation jobs	TCP (SSL) to the database; model push to inference server
SMS / E-mail Provider (external)	Delivers notifications to cardholders	HTTPS

Result:- The deployment diagram showing the hardware nodes and their communication paths was drawn.